MIT | **Academy ofi Engineering**

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Subject (Code):
Essentials ofi Data Science (2304102L)

EDS CodeTantra Programs

Name	(PRN)	Div-Batch
Pruthviraj Satish Bankar	(202501110210)	AL2-A25

Practical 01

1.1. Practice Lab Assignments

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula: $p = m\ v$

where:

p is the mass of the object (in kilograms).

m is the velocity of the object (in meters per second).

Input Format:

A single floating-point number representing the mass of the object in kilograms.

A single floating-point number representing the velocity of the object in meters per second.

Output Format:

The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

```
# Read m and v
m = float(input())
v = float(input())

# Calculate momentum and display
p = m * v
print(f"{p:.2f}kgm/s")
```

1.1.2 Conditional Calculation based on the Number of Digits

Write a Python program that accepts an integer as input. Depending on the number of digits in n.

Constraints:

$1 \leq n \leq 999$

Input Format:

The input consists of a single integer n.

Output Format:

If n is a single-digit number, print its square.

If n is a two-digit number, print its square root (rounded to two decimal places).

If n is a three-digit number, print its cube root (rounded to two decimal places).

Else print "Invalid".

```
n = int(input())

# Check for "Invalid" boundaries first to simplify the rest of the logic
if n < 1 or n  = 1000:
   print("Invalid")
elif n < 10:
   # We know n  = 1 because the first check failed
   print(n**2)
elif n < 100:
   # We know n  = 10 because the previous check failed
   print(f"{n**0.5:.2f}")
else:
   # We know n  = 100 and n < 1000 because all previous checks failed
   print(f"{n**(1/3):.2f}")
```

1.1.3. Days Between Two Dates

Write a Python program to calculate the number of days between two dates.

Input Format:

The first line contains the first date in the format *YYYY-MM-DD*.

The second line contains the second date in the format *YYYY-MM-DD*.

Output Format:

An integer representing the number of days between the given dates.

```
from datetime import date

def calculate_days(first_date_str, second_date_str):
    # Convert the string inputs into date objects
    # fromisoformat handles 'YYYY-MM-DD' automatically
    d1 = date.fromisoformat(first_date_str)
    d2 = date.fromisoformat(second_date_str)

    # Calculate the difference between the dates
    delta = d2 - d1

    return delta


# Read the dates as standard input
date1_str = input().strip()
date2_str = input().strip()

# Print the absolute value of the days to ensure a positive integer
print(abs(calculate_days(date1_str, date2_str).days))
```

1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

```
n = int(input())
reversed_num = 0

while n > 0:
    digit = n % 10              # Get the last digit
    reversed_num = reversed_num * 10 + digit  # Add digit to the new number
    n = n  / 10                 # Remove the last digit from n

print(reversed_num)
```

Aliter: The most Pythonic Way —

```
# Read the input as an integer
num = int(input())

# Convert to string, reverse it with slicing [ :-1], and convert back to int
reversed_num = int(str(num)[ :-1])

print(reversed_num)
```

1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number.
  <number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

```
num = int(input())

for i in range(1, 11):
   print(f"{num} x {i} = {num * i}")
```

1.2 Lab Assignments

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n, the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

```
# Read the number of courses
n = int(input())

# Read the marks as a list of integers
marks = list(map(int, input().split()))
```

```
# Check if the student failed any course (marks < 40)
if any(m < 40 for m in marks):
    print("Fail")
else:
    # Calculate aggregate percentage
    aggregate_percentage = sum(marks) / n

    # Determine the grade based on the percentage
    if aggregate_percentage > 75:
        grade = "Distinction"
    elif 60  = aggregate_percentage < 75:
        grade = "First Division"
    elif 50  = aggregate_percentage < 60:
        grade = "Second Division"
    elif 40  = aggregate_percentage < 50:
        grade = "Third Division"
    # Note: Logic implies if percentage < 40 it would fail,
    # but the initial check covers individual failures.

    # Print the results
    print(f"Aggregate Percentage: {aggregate_percentage:.2f}")
    print(f"Grade: {grade}")
```

1.2.2. Fibonacci Series using Recursive Function

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

A single integer n representing the number of terms of generate.

Output Format:

A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

```
def fibonacci(n):
    if n  = 2:
        return n-1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
```

```
n = int(input())
for i in range(1, n + 1):
   print(fibonacci(i), end=" ")
```

1.2.3. Pattern-1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Test case 1:

```
4
```

```
* ⏎

* * ⏎

* * * ⏎

* * * * ⏎
```

```
n = int(input())
for i in range(1, n + 1):
   print("* " * i)
```

1.2.4. Pattern-2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Test case 1:

4

1 ·⏎

1 ·2 ·⏎

1 ·2 ·3 ·⏎

1 ·2 ·3 ·4 ·⏎

```
n = int(input())
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print("")
```

Practical 02

2.1. Practice Lab Assignments

2.1.1. List Operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove**: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display**: Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit**: Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice".Ensure that the program continues to prompt the user until they choose to quit (option 4).

Test Case 1

```
1. Add⏎
2. Remove⏎
3. Display⏎
4. Quit⏎
Enter choice: 1
Integer: 11
List after adding: [11]⏎
1. Add⏎
2. Remove⏎
3. Display⏎
4. Quit⏎
Enter choice: 1
Integer: 25
List after adding: [11, 25]⏎
1. Add⏎
2. Remove⏎
3. Display⏎
4. Quit⏎
Enter choice: 2
Integer: 26
Element not found⏎
1. Add⏎
2. Remove⏎
3. Display⏎
4. Quit⏎
Enter choice: 2
Integer: 11
List after removing: [25]⏎
1. Add⏎
2. Remove⏎
3. Display⏎
4. Quit⏎
Enter choice: 2
Integer: 25
List after removing: []⏎
```

```
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
List is empty
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
List is empty
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 8
Invalid choice
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 4
```

```python
def main():
    my_list = []

    while True:
        # Display the Menu
        print("1. Add")
        print("2. Remove")
        print("3. Display")
        print("4. quit")

        # Prompt for entering the choice
        choice = input("Enter choice: ")

        if choice  = "1": # add
            try:
                val = int(input("Integer: "))
                my_list.append(val)
                print(f"List after adding: {my_list}")
            except ValueError:
                print("Invalid input")

        elif choice  = "2": # Remove
            # Check if list is empty before asking for input
            if not my_list:
                print("List is empty")
            else:
                try:
                    val = int(input("Integer: "))
                    if val in my_list:
                        my_list.remove(val)
                        print(f"List after removing: {my_list}")
                    else:
                        print("Element not found")
                except ValueError:
                    print("Invalid input")
```

```
        elif choice  = "3": # Display
            if not my_list:
                print("List is empty")
            else:
                print(my_list)

        elif choice  = "4": # Quit
            break

        else:
            print("Invalid choice")


if  _name _  = " _main _":
    main()
```

2.1.2. Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

```
Original·Dictionary:·{1:·'Amit',·2:·'Riya',·3:·'Kiran',·4:·'Neha',·5:·'A
rjun',·6:·'Pooja',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali'}⏎

11
Suresh
After·Insertion:·{1:·'Amit',·2:·'Riya',·3:·'Kiran',·4:·'Neha',·5:·'Arjun
',·6:·'Pooja',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali',·11:·'S
uresh'}⏎
3
Karthik
After·Update:·{1:·'Amit',·2:·'Riya',·3:·'Karthik',·4:·'Neha',·5:·'Arjun'
,·6:·'Pooja',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali',·11:·'Su
resh'}⏎
5
After·Deletion:·{1:·'Amit',·2:·'Riya',·3:·'Karthik',·4:·'Neha',·6:·'Pooj
a',·7:·'Rahul',·8:·'Sneha',·9:·'Vikram',·10:·'Anjali',·11:·'Suresh'}⏎
Traversing·Dictionary:⏎
1·:·Amit⏎
2·:·Riya⏎
3·:·Karthik⏎
4·:·Neha⏎
6·:·Pooja⏎
```

```
7 : Rahul
8 : Sneha
9 : Vikram
10 : Anjali
11 : Suresh


# Initial dictionary with 10 predefined records
student = {
    1: "Amit",
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}

def main():
    # 1. Display Original Dictionary
    print(f"Original Dictionary: {student}")

    # 2. Insertion Operation
    try:
        insert_key = int(input())
        insert_value = input()
        student[insert_key] = insert_value
    except (ValueError, EOFError):
        pass
    print(f"After Insertion: {student}")

    # 3. Update Operation
    try:
        update_key = int(input())
        update_value = input()
        if update_key in student:
            student[update_key] = update_value
    except (ValueError, EOFError):
        pass
    print(f"After Update: {student}")

    # 4. Deletion Operation
    try:
        delete_key = int(input())
```

```
        if delete_key in student:
            del student[delete_key]
    except (ValueError, EOFError):
        pass
    print(f"After Deletion: {student}")

    # 5. Traversal Operation
    print("Traversing Dictionary:")
    for key, value in student.items():
        print(f"{key} : {value}")

if _name_ = "_main_":
    main()
```

2.2. Lab Assignments

2.2.1. Linear Search Technique

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index.
- If the element is not found, print **Not found**.

Sample Test Case:

Input:	**Output:**
1 2 3 4 3 5 6 3	2

```
def main():
    # Read the input line, split by spaces and convert each part into an integer
    numbers = list(map(int, input().split()))

    # Read the key element to be searched
    key = int(input())

    # Linear Search Logic
    found = False
    for i in range(len(numbers)):
        if numbers[i]  = key:
            print(i)  # Print the index of the first occurrence
            found = True  # Mark as found
            break  # Stop searching once found

    if not found:
        print("Not found")


if  _name _  = " _main _":
    main()
```

Aliter: The most Pythonic Way —

```
numbers = list(map(int, input().split()))
key = int(input())

if key in numbers:
   print(numbers.index(key))
else:
   print("Not found")
```

2.2.2. Captain of the Team

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

```
# Read the input line, split by spaces, and convert each part to an
integer
heights = list(map(int, input().split()))

# Find the maximum value in the list
tallest_player = max(heights)

# Print the result
print(tallest_player)
```

Practical 03

3.1 Practice Lab Assignments

3.1.1. Numpy Array Operations

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using ndim
- The shape of the array using shape
- The total number of elements in the array using size

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, *r* and *c*, representing the number of rows and the number of columns.
- The next *r* lines contain *c* space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use reshape() function to reshape the input array with the specified number of rows and columns.

```
import numpy as np

# Read the number of rows and columns
r, c = map(int, input().split())

# Read the array elements row by row and flatten them into a single list
elements = []
for _ in range(r):
    row = list(map(int, input().split()))
    elements.extend(row)
```

```
# Create a NumPy array from the flattened list
arr = np.array(elements)

# Reshape the array to the specified dimensions (r rows, c columns)
reshaped_arr = arr.reshape(r, c)

# Display the required outputs
print(reshaped_arr)
print(reshaped_arr.ndim)     #Returns the number of dimensions (2 for a
matrix)
print(reshaped_arr.shape)    #Returns a tuple representing the dimensions
(rows, columns)
print(reshaped_arr.size)     #Returns the total count of elements in the
array
```

3.2 Lab Assignments

3.2.1. NumPy: Matrix Operations

The given code takes two 3×3 matrices, *matrix_a*, and *matrix_b*, as input from the user and converts them into NumPy arrays.

Task: You are required to compute and display the results of the following matrix operations:

1. **Addition** (*matrix_a* + *matrix_b*)
2. **Subtraction** (*matrix_a* - *matrix_b*)
3. **Element-wise Multiplication** (*matrix_a* * *matrix_b*)
4. **Matrix Multiplication** (*matrix_a* • *matrix_b*)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for *matrix_a*, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for *matrix_b*, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

```
import numpy as np

# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

# Addition
print("Addition (A + B):")
a = matrix_a + matrix_b
```

```
print(a)

# Subtraction
print("Subtraction (A - B):")
b = matrix_a - matrix_b
print(b)

# Multiplication (element-wise)
print("Element-wise Multiplication (A * B):")
m = matrix_a*matrix_b
print(m)

# Matrix multiplication (dot product)
print("A dot B:")
d = np.dot(matrix_a,matrix_b)
print(d)

# Transpose
print("Transpose of A:")
t = np.transpose(matrix_a)
print(t)
```

3.2.2. NumPy Horizontal and Vertical Stacking of Arrays

You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

```
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])


# Perform horizontal stacking (hstack)
h_stack = np.hstack((arr1, arr2))


# Perform vertical stacking (vstack)
v_stack = np.vstack((arr1, arr2))


# Display the results
print("Horizontal Stack:")
print(h_stack)

print("Vertical Stack:")
print(v_stack)
```

3.2.3. NumPy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.

- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

```
import numpy as np

# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())

# Generate the sequence using np.arange()
arr = np.arange(start,stop,step)

# Print the generated sequence
print(arr)
```

3.2.4. NumPy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators

You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A_i OR B_i).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

```
import numpy as np


def array_operations(A, B):
    # Convert A and B to NumPy arrays
    arr_A = np.array(A)
    arr_B = np.array(B)

    # Arithmetic Operations
    sum_result = np.add(arr_A, arr_B)
    diff_result = np.subtract(arr_A, arr_B)
    prod_result = np.multiply(arr_A, arr_B)

    # Statistical Operations
    mean_A = np.mean(arr_A)
    median_A = np.median(arr_A)
    std_dev_A = np.std(arr_A)

    # Bitwise Operations
    and_result = np.bitwise_and(arr_A, arr_B)
    or_result = np.bitwise_or(arr_A, arr_B)
    xor_result = np.bitwise_xor(arr_A, arr_B)

    # Output results with one space between each element
    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
    print("Element-wise Product:", ' '.join(map(str, prod_result)))

    print(f"Mean of A: {mean_A}")
    print(f"Median of A: {median_A}")
```

```
    print(f"Standard Deviation of A: {std_dev_A}")

    print("Bitwise AND:", ' '.join(map(str, and_result)))
    print("Bitwise OR:", ' '.join(map(str, or_result)))
    print("Bitwise XOR:", ' '.join(map(str, xor_result)))


A = list(map(int, input().split()))  # Elements of array A
B = list(map(int, input().split()))  # Elements of array B
array_operations(A, B)
```

3.2.5. NumPy: Copying and Viewing Array

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the **original_array** and assigning it to **view_array**.
- Creating a copy of the **original_array** and assigning it to **copy_array**.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:
  Original array after modifying view: <original_array>
  View array: <view_array>

- After modifying the copy:
  Original array after modifying copy: <original_array>
  Copy array: <copy_array>

```
import numpy as np

inputlist = list(map(int,input().split(" ")))

# Original array
```

```
original_array = np.array(inputlist)

# Create a view of the original_array
view_array = original_array.view()

# Create a copy of the original_array
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

3.2.6. NumPy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, **array1**, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- **search_value**: The value to search for in the array.
- **count_value**: The value to count its occurrences in the array.
- **broadcast_value**: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. Searching: Find the indices where **search_value** appears in **array1** and print these indices.

2. Counting: Count how many times **count_value** appears in **array1** and print the count.

3. Broadcasting: Add **broadcast_value** to each element of **array1** using broadcasting, and print the resulting array.

4. Sorting: Sort **array1** in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing **array1**.
2. An integer **search_value** represents the value to search for in the array.
3. An integer **count_value** represents the value to count in the array.
4. An integer **broadcast_value** represents the value to add to each element of the array.

Output Format:

1. The indices where **search_value** occurs in **array1**.
2. The count of occurrences of **count_value** in **array1**.
3. The array after adding the **broadcast_value** to each element.
4. The sorted array.

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split())))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))

# Find indices where value matches in array1
indices = np.where(array1  = search_value)[0]
print(indices)

# Count occurrences in array1
count = np.count_nonzero(array1  = count_value)
print(count)

# Broadcasting addition
broadcasted_array = array1 + broadcast_value
print(broadcasted_array)

# Sort the first array
sorted_array = np.sort(array1)
print(sorted_array)
```

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.

- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored >=90 in each subject:** Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored >=90:** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.
    - Note: There is a Ghost Table in the Test Case. Print the original array to pass it.
- **Find index positions of students who scored 79 in Subject 1:** Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

```
import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

# 1. Print all student details
print("All student Details:\n", a)

# 2. Find total students
print("Total Students:", int(a.shape[0]))

# 3. Print all student roll numbers
print("All Student Roll Nos", a[:, 0])

# 4. Print Subject 1 marks
print("Subject 1 Marks", a[:, 1])

# 5. Find minimum marks in Subject 2
print("Min marks in Subject 2", np.min(a[:, 2]))

# 6. Find maximum marks in Subject 3
print("Max marks in Subject 3", np.max(a[:, 3]))

# 7. Print all subject marks
print("All subject marks:", a[:, 1:])

# 8. Find total marks of students
```

```
print("Total Marks", np.sum(a[:, 1:], axis=1))

# 9. Find the average marks of each student
# Rounding to 1 decimal place matches the sample output format
print(np.round(np.mean(a[:, 1:], axis=1), 1))

# 10. Find average marks of each subject
print("Average Marks of each subject", np.mean(a[:, 1:], axis=0))

# 11. Find average marks of Subject 1 and Subject 2
print("Average Marks of S1 and S2", np.mean(a[:, 1:3], axis=0))

# 12. Find average marks of Subject 1 and Subject 3
print("Average Marks of S1 and S3", np.mean(a[:, [1, 3]], axis=0))

# 13. Find the roll number of the student with maximum marks in Subject3
max_s3_index = np.argmax(a[:, 3])
print("Roll no who got maximum marks in Subject 3", a[max_s3_index, 0])

# 14. Find the roll number of the student with minimum marks in Subject2
min_s2_index = np.argmin(a[:, 2])
print("Roll no who got minimum marks in Subject 2", a[min_s2_index, 0])

# 15. Find the roll number of students who scored 24 marks in Subject 2
# We use slicing [:, [0]] to maintain the 2D structure as shown in the
sample output [[310.]]
roll_24 = a[a[:, 2]  = 24][:, [0]]
print("Roll no who got 24 marks in Subject 2", roll_24)

# 16. Find the count of students who got less than 40 marks in Subject 1
count_s1_less_40 = np.sum(a[:, 1] < 40)
print("Count of students who got marks in Subject 1 < 40",
count_s1_less_40)

# 17. Find the count of students who got more than 90 marks in Subject 2
count_s2_more_90 = np.sum(a[:, 2] > 90)
print(f"Count of students who got marks in Subject 2 > 90:
{count_s2_more_90}")

# 18. Find the count of students who scored  =90 in each subject
count_each_sub_90 = np.sum(a[:, 1:]  = 90, axis=0)
print("Count of students in each subject who got marks  = 90:",
count_each_sub_90)

# 19. Find the count of subjects in which each student scored  =90
# First print Roll numbers as per sample output
print("Roll no:", a[:, 0])
```

```
count_stud_sub_90 = np.sum(a[:, 1:]  = 90, axis=1)
print("Count of subjects in which student got marks  = 90:",
count_stud_sub_90)

# 20. Print Subject 1 marks in ascending order
print(np.sort(a[:, 1]))

# 21. Print students who scored between 50 and 90 in Subject 1
# Filtering rows where Subject 1 (column 1) is  = 50 and  = 90
students_50_90 = a[(a[:, 1]  = 50) & (a[:, 1]  = 90)]
print(students_50_90)

#Ghost Table: Print the Original Array to pass it
print(a)

# 22. Find index positions of students who scored 79 in Subject 1
indices_79 = np.where(a[:, 1]  = 79)
print(indices_79)
```

Practical 04

4.1. Practice Lab Assignments

4.1.1. Pandas – Series Creation and Manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the **groupby** and **mean()** operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

```
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
series = pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series%2 =0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in
grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

4.1.2. Dictionary to DataFrame

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:
- Convert the dictionary to a Pandas DataFrame.

Add a new row:
- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:
- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:
- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:
- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:
- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:
- Remove the **Age** column.
- Display the DataFrame after deleting the column.

```
import pandas as pd

# Provided dictionary of lists
data = {
   'Name': ['Alice', 'Bob', 'Charlie'],
   'Age': [25, 30, 35],
}

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new row
new_name = input("New name: ")
new_age = int(input("New age: "))
new_row = {'Name': new_name, 'Age': new_age }
df = df.append(new_row,ignore_index = True)
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
modify_index = int(input("Index of row to modify: "))
new_age = int(input("New age: "))
df.at[modify_index,'Age'] = new_age
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

# Deleting a row
del_index = int(input("Index of row to delete: "))
df = df.drop(del_index).reset_index(drop = True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

# Adding a new column
genders = input("Enter genders separated by space: ").split()
df["Gender"] = genders
# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
df['Name'] = df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")
print(df)

# Deleting a column
df = df.drop(columns = 'Age').reset_index(drop = True)
# Display the DataFrame after deleting a column
print("After deleting a column:")
print(df)
```

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students(limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold(consider the threshold grade is **'B'**).

Note:
Refer to the displayed test cases for better understanding.

```
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
"Grade"])

# Display the first five rows
print("First five rows:")
print(data.head())

# Calculate the average age.
average_age = data['Age'].mean()
print(f"Average age: {round(average_age, 2)}")

# Filter students with a grade up to 'B', i.e., 'Grade'  = 'B'
filtered_students = data[data['Grade']  = 'B']
print("Students with a grade up to B")
print(filtered_students)
```

4.2. Lab Assignments

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```python
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# Calculate the total sales for each transaction (Quantity * Price)
df['Total Sales'] = df['quantity'] * df['Price']

# Convert the 'Date' column to datetime objects to extract month information
df['Date'] = pd.to_datetime(df['Date'])
```

```
# Create a new column for the Month name
df['Month'] = df['Date'].dt.to_period('M')

# Group by Month and sum the Total Sales
monthly_sales = df.groupby('Month')['Total Sales'].sum()

# Find the best month (index with the max sales value) and the sales
figure
best_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()

# Display the results
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```python
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# Group by Product and calculate the total quantity sold for each
product_quantity = df.groupby('Product')['quantity'].sum()

# Find the most sold product (index with the max total quantity value)
and the total quantity sold of that product
best_product = product_quantity.idxmax()
highest_quantity = product_quantity.max()

# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
```

2025-01-05,Product C,10,30,Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# Group by City and sum the Quantity column
city_sales = df.groupby('City')['quantity'].sum()

# Find the best city (index with the max total quantity value)
best_city = city_sales.idxmax()

# Display the result
print(f"City sold the most products: {best_city}")
```

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
```

```
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B**: Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C**: Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C**: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```python
import pandas as pd
from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)

# 1. Group the data by 'Date' and get the list of products for each date
daily_transactions = df.groupby('Date')['Product'].apply(list)

# Initialize a Counter to store the frequency of each pair
pair_counts = Counter()

# 2. Iterate through each date's transaction
for products in daily_transactions:
```

```
    # Sort the products to ensure (A, B) is treated the same as (B, A)
    products = sorted(products)

    # Generate all possible pairs (combinations of size 2)
    # If a date has fewer than 2 products, this produces nothing
(correctly)
    pairs = list(combinations(products, 2))

    # Update the counter with these pairs
    pair_counts.update(pairs)

# 3. Find the maximum frequency count and print all pairs that match
this maximum frequency
if pair_counts:
    max_count = max(pair_counts.values())

    for pair, count in pair_counts.items():
        if count  = max_count:
            print(f"{pair[0]} and {pair[1]}: {count} times")

else:
    print("No product pairs found.")
```

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.

10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# 1. Display the first 5 rows of the dataset
print(data.head())

# 2. Display the last 5 rows of the dataset
print(data.tail())

# 3. Get the shape of the dataset
print(data.shape)

# 4. Get a summary of the dataset (info)
print(data.info())
```

```
# 5. Get basic statistics of the dataset
print(data.describe())

# 6. Check for missing values
print(data.isnull().sum())

# 7. Fill missing values in the 'Age' column with the median age
data['Age'].fillna(data['Age'].median(), inplace = True)

# 8. Fill missing values in the 'Embarked' column with the mode
data['Embarked'].fillna(data['Embarked'].mode()[0],inplace=True)

# 9. Drop the 'Cabin' column due to many missing values
data.drop(columns='Cabin',inplace = True)

# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
data["FamilySize"] = data['SibSp']+data['Parch']
```

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']

# Create a new column 'IsAlone' which is 1 if the passenger is alone (i.e., FamilySize = 0), otherwise 0
data['IsAlone'] = np.where(data['FamilySize']  = 0, 1, 0)
# data['IsAlone'] = (data['FamilySize']  = 0).astype(int) # Alternate way

# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns=['Embarked'])

# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print( mean_age)

# 5. Get the median fare of passengers
median_fare = data['Fare'].median()
print( median_fare)
```

```
# 6. Get the number of passengers by class
print( data['Pclass'].value_counts())

# 7. Get the number of passengers by gender
print( data['Sex'].value_counts())

# 8. Get the number of passengers by survival status
print( data['Survived'].value_counts())

# 9. Calculate the overall survival rate
survival_rate = data['Survived'].mean()
print(format(survival_rate))

# 10. Calculate the survival rate by gender
survival_by_gender = data.groupby('Sex')['Survived'].mean()
print( survival_by_gender)
```

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class
print(data.groupby('Pclass')['Survived'].mean())

# 2. Calculate the survival rate by embarkation location (Embarked_S)
print(data.groupby('Embarked_S')['Survived'].mean())

# 3. Calculate the survival rate by family size
print(data.groupby('FamilySize')['Survived'].mean())

# 4. Calculate the survival rate by being alone
print(data.groupby('IsAlone')['Survived'].mean())

# 5. Get the average fare by passenger class
print(data.groupby('Pclass')['Fare'].mean())

# 6. Get the average age by passenger class
```

```
print(data.groupby('Pclass')['Age'].mean())

# 7. Get the average age by survival status
print(data.groupby('Survived')['Age'].mean())

# 8. Get the average fare by survival status
print(data.groupby('Survived')['Fare'].mean())

# 9. Get the number of survivors by class
# Filter for Survived=1, then count occurrences of Pclass
print(data[data['Survived']  = 1]['Pclass'].value_counts())

# 10. Get the number of non-survivors by class
# Filter for Survived=0, then count occurrences of Pclass
print(data[data['Survived']  = 0]['Pclass'].value_counts())
```

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)


# 1. Get the number of survivors by gender
print(data[data['Survived'] =1]['Sex'].value_counts())

# 2. Get the number of non-survivors by gender
print(data[data['Survived'] =0]['Sex'].value_counts())

# 3. Get the number of survivors by embarked location
print(data[data['Survived'] =1]['Embarked_S'].value_counts())

# 4. Get the number of non-survivors by embarked location
print(data[data['Survived'] =0]['Embarked_S'].value_counts())

# 5. Calculate the percentage of children (Age < 18) who survived
print(data[data['Age']<18]['Survived'].mean())

# 6. Calculate the percentage of adults (Age  = 18) who survived
print(data[data['Age'] =18]['Survived'].mean())
```

```python
# 7. Get the median age of survivors
print(data[data['Survived'] =1]['Age'].median())

# 8. Get the median age of non-survivors
print(data[data['Survived'] =0]['Age'].median())

# 9. Get the median fare of survivors
print(data[data['Survived'] =1]['Fare'].median())

# 10. Get the median fare of non-survivors
print(data[data['Survived'] =0]['Fare'].median())
```

Practical 05

5.1 Practice Lab Assignments

5.1.1. Stacked Plot

Create a stacked area plot to visualize the temperature variations for three different cities (City A, City B, and City C) across the months of the year. The temperature data is provided for each city in the editor.

Your task is to:

- Create a stacked area plot using the data.
- Label the x-axis as "Month", the y-axis as "Temperature", and provide the title "Temperature Variation" for the plot.
- Display the plot showing the temperature variation for each city throughout the months of the year.

```
import matplotlib.pyplot as plt
import pandas as pd

# Data for Months and Temperature for three cities
data = {
    'Month': ['January', 'February', 'March', 'April', 'May', 'June',
'July', 'August', 'September', 'October', 'November', 'December'],
    'City_A_Temperature': [5, 7, 10, 13, 17, 20, 22, 21, 18, 12, 8, 6],
    'City_B_Temperature': [2, 3, 5, 6, 10, 14, 16, 17, 12, 9, 5, 3],
    'City_C_Temperature': [3, 4, 6, 8, 9, 12, 15, 14, 10, 7, 4, 2]
}

# Create a DataFrame
df = pd.DataFrame(data)

# Create the stacked area plot
plt.stackplot(df['Month'], df['City_A_Temperature'],
df['City_B_Temperature'], df['City_C_Temperature'])

# Add labels and title
plt.xlabel('Month')
plt.ylabel('Temperature')
plt.title('Temperature Variation')

# Display the plot
plt.show()
```

5.2 Lab Assignments

5.2.1. Titanic Dataset

Write a Python program to analyze and visualize data from the Titanic dataset based on the following instructions:

Dataset Information:
The dataset is stored in a CSV file named titanic.csv and has been loaded using the pandas library. It contains the following columns:

- Pclass: Passenger class (1 = First, 2 = Second, 3 = Third).
- Gender: Gender of the passenger (male/female).
- Age: Age of the passenger.
- Survived: Survival status (0 = Did not survive, 1 = Survived).
- Fare: Ticket fare paid by the passenger.

Visualization:
To represent these trends, you will create 5 visualizations using Matplotlib. The visualizations should be arranged in a 3x2 grid (3 rows and 2 columns).

Visualization Details:
Write the code to create a series of visualizations as follows:

Bar Plot (Pclass Distribution):
- Create a bar plot to show the distribution of passengers across the different passenger classes (Pclass).
- Use the color skyblue for the bars.
- Title the plot as **"Passenger Class Distribution"**.
- Label the x-axis as **"Pclass"** and the y-axis as **"Count"**.

Pie Chart (Gender Distribution):
- Create a pie chart to display the distribution of male and female passengers.
- Use lightblue for males and lightcoral for females.
- Include percentages on the slices (use autopct='%1.1f%%').
- Title the plot as **"Gender Distribution"**.

Histogram (Age Distribution):
- Create a histogram to visualize the distribution of passengers' ages.
- Use lightgreen for the bars with black edges (edgecolor = 'black').
- Set the number of bins to **8** for the histogram.
- Title the plot as **"Age Distribution"**.
- Label the x-axis as **"Age"** and the y-axis as **"Frequency"**.

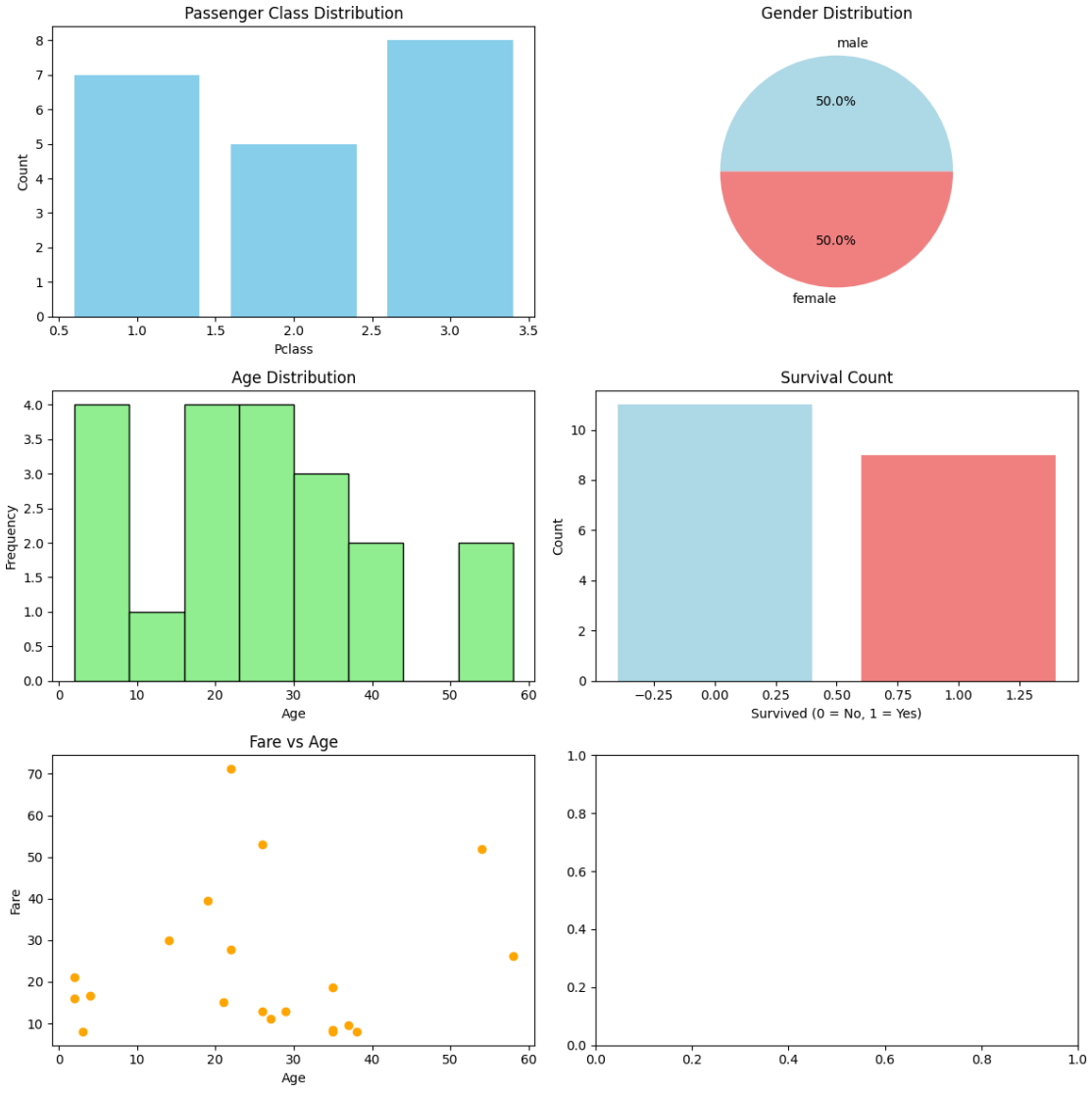
Bar Plot (Survival Count):

- Create a bar plot to show the count of passengers who survived and those who did not, based on the Survived column.
- Use the colors lightblue for survivors (1) and lightcoral for non-survivors (0).
- Title the plot as **"Survival Count"**.
- Label the x-axis as **"Survived (0 = No, 1 = Yes)"** and the y-axis as **"Count"**.

Scatter Plot (Fare vs Age):

- Create a scatter plot to visualize the relationship between the Fare and Age of passengers.
- Use orange for the data points.
- Title the plot as **"Fare vs Age"**.
- Label the x-axis as **"Age"** and the y-axis as **"Fare"**.

Note: Refer to the displayed plot in the sample test cases for better understanding.



```python
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset from the CSV file
df = pd.read_csv('titanic.csv')

# Set up the figure for 5 subplots
fig, axes = plt.subplots(3, 2, figsize=(12, 12))

#   - 1. Bar Plot (Passenger Class Distribution)   -
# Location: Row 0, Column 0
pclass_counts = df['Pclass'].value_counts().sort_index()
axes[0, 0].bar(pclass_counts.index, pclass_counts.values,
color='skyblue')
axes[0, 0].set_title("Passenger Class Distribution")
axes[0, 0].set_xlabel("Pclass")
axes[0, 0].set_ylabel("Count")


#   - 2. Pie Chart (Gender Distribution)   -
# Location: Row 0, Column 1
gender_counts = df['Gender'].value_counts()
axes[0, 1].pie(gender_counts, labels=gender_counts.index,
autopct='%1.1f%%', colors=['lightblue', 'lightcoral'])
axes[0, 1].set_title("Gender Distribution")


#   - 3. Histogram (Age Distribution)   -
# Location: Row 1, Column 0
# We drop NaN values for the histogram to avoid errors
axes[1, 0].hist(df['Age'].dropna(), bins=8, color='lightgreen',
edgecolor='black')
axes[1, 0].set_title("Age Distribution")
axes[1, 0].set_xlabel("Age")
axes[1, 0].set_ylabel("Frequency")


#   - 4. Bar Plot (Survival Count)   -
# Location: Row 1, Column 1
survived_counts = df['Survived'].value_counts().sort_index()
axes[1, 1].bar(survived_counts.index, survived_counts.values,
color=['lightblue', 'lightcoral'])
axes[1, 1].set_title("Survival Count")
axes[1, 1].set_xlabel("Survived (0 = No, 1 = Yes)")
axes[1, 1].set_ylabel("Count")
```

```
#   - 5. Scatter Plot (Fare vs Age)   -
# Location: Row 2, Column 0
axes[2, 0].scatter(df['Age'],df['Fare'], color='orange',
edgecolors='black')
axes[2, 0].set_title("Fare vs Age")
axes[2, 0].set_xlabel("Age")
axes[2, 0].set_ylabel("Fare")


#   - 6. Empty Plot   -
# axes[2, 1].axis('off') # This fails the test case so keep it commented


# Adjust layout to prevent overlap and ensure everything fits nicely
plt.tight_layout()

# Display the plot
plt.show()
```

5.2.2. Histogram of Passenger Information of Titanic

Write a Python code to plot a histogram for the distribution of the 'Age' column from the Titanic dataset. The histogram should display the frequency of different age ranges with the following specifications:

1. Use **30 bins** for the histogram.
2. Set the **edge color** of the bars to **black** (k).
3. Label the x-axis as **'Age'** and the y-axis as **'Frequency'**.
4. Add the title **"Age Distribution"** to the histogram.

The Titanic dataset contains columns as shown below,

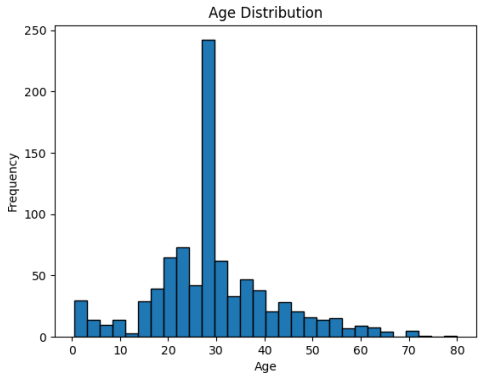
PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Create the Histogram
plt.hist(data["Age"].dropna(), bins=30, edgecolor= "k")

# Add labels and title
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.title("Age Distribution")

# Display the plot
plt.show()
```

5.2.3. Bar Plot of Survival rate of Passengers

Write a Python code to plot a bar chart that shows the count of passengers who survived and did not survive in the Titanic dataset. The chart should display the following specifications:

1. Use the **'Survived'** column to show the count of survivors (0 = Did not survive, 1 = Survived).
2. Set the chart type to **'bar'**.
3. Add the title **"Survival Count"** to the chart.
4. Label the x-axis as **'Survived'** and the y-axis as **'Count'**.

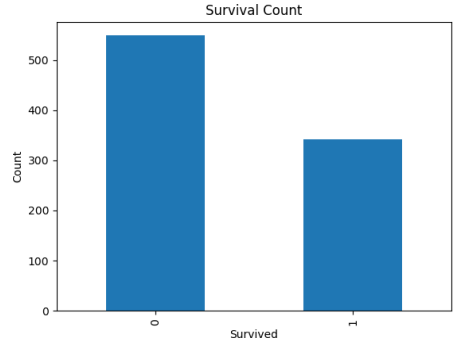
The Titanic dataset contains columns as shown below,

PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.



```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Count the number of survivors and non-survivors
survival_counts = data['Survived'].value_counts().sort_index()

# Create the bar plot
survival_counts.plot(kind='bar')

# Add title and labels
plt.title("Survival Count")
plt.xlabel("Survived")
plt.ylabel("Count")

# Display the plot
plt.show()
```

5.2.4. Bar Plot for Survival by Gender

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by gender, in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the **'Sex'** column, then use the **value_counts()** function to count the occurrences of survivors (0 = Did not survive, 1 = Survived) for each gender.
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title **"Survival by Gender"** to the chart.
4. Label the x-axis as **'Gender'** and the y-axis as **'Count'**.
5. The legend should indicate **'Not Survived'** and **'Survived'**.

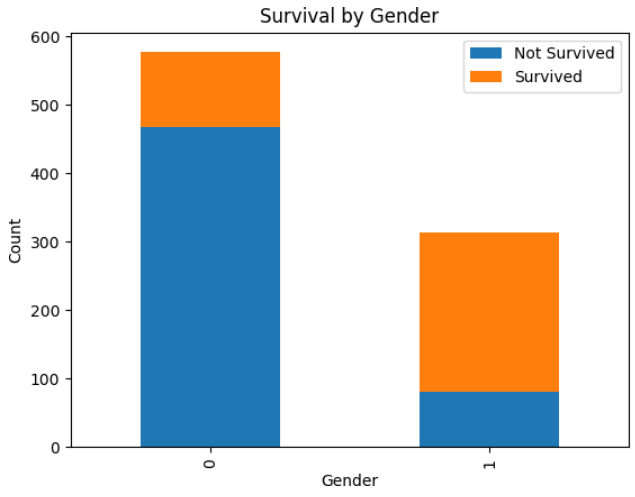
The Titanic dataset contains columns as shown below,

PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Prepare the data. Group by Sex and Survived, then count the size of
each group and unstack to get columns for 0 and 1
survival_gender = data.groupby(['Sex', 'Survived']).size().unstack()

# Create the stacked bar chart
ax = survival_gender.plot(kind='bar', stacked=True)

# Add title and labels
plt.title("Survival by Gender")
plt.xlabel("Gender")
plt.ylabel("Count")

# Customize Legend
plt.legend(["Not Survived", "Survived"])

# Display the plot
plt.show()
```

5.2.5. Bar Plot for Survival by Class

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by passenger class (**Pclass**), in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the **Pclass** column and count the number of survivors (0 = Did not survive, 1 = Survived) for each class using **value_counts()**.
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title **"Survival by Pclass"** to the chart.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Count'**.
5. The legend should indicate **'Not Survived'** and **'Survived'**.

The Titanic dataset contains columns as shown below,

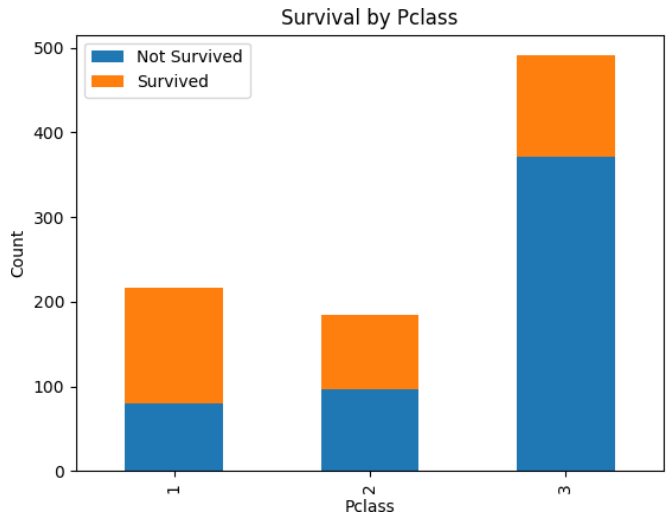
PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note:

- Refer to the visible test case for better reference.
- Ensure you use the **groupby()** function with **value_counts()** to count the survivors and non-survivors for each **Pclass**.
- Do **not** manually use **size()** or **unstack()** without **value_counts()**. Use the **value_counts()** method for counting survival status directly.




```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
```

```
# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Prepare the data. Group by Pclass and Survived, count the size, and
unstack to get columns for 0 and 1
survival_pclass = data.groupby(['Pclass', 'Survived']).size().unstack()

# Create the stacked bar chart
survival_pclass.plot(kind='bar', stacked=True)

# Add title and labels
plt.title("Survival by Pclass")
plt.xlabel("Pclass")
plt.ylabel("Count")

# Customize Legend
plt.legend(["Not Survived", "Survived"])

# Display the plot
plt.show()
```

5.2.6. Bar Plot for Survival by Embarked

Write a Python code to plot a stacked bar chart showing the survival count for passengers based on their embarkation location in the Titanic dataset.

The chart should display the following specifications:

1. Use the **Embarked** column to determine the embarkation location. After converting this column into dummy variables (using **pd.get_dummies()**), plot the survival count based on the **Embarked_Q** column (representing passengers who embarked from Queenstown) in relation to survival.
2. Set the chart type to 'bar' and make it stacked.
3. Add the title "**Survival by Embarked** " to the chart.
4. Label the x-axis as '**Embarked**' and the y-axis as '**Count**'.

5. Include a legend to distinguish between survivors and non-survivors (label the legend as **'Survived'** and **'Not Survived'**).

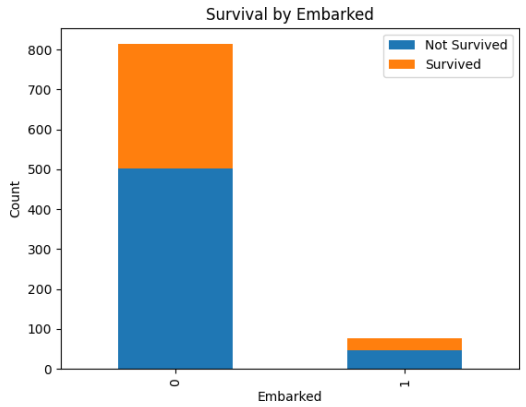
The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Prepare the data. Group by 'Embarked_Q' and 'Survived', count the
size, and unstack
survival_embarked = data.groupby(['Embarked_q',
'Survived']).size().unstack()

# Create the stacked bar chart
survival_embarked.plot(kind='bar', stacked=True)

# Add title and labels
plt.title("Survival by Embarked")
plt.xlabel("Embarked")
plt.ylabel("Count")

# Customize Legend 0 = Not Survived, 1 = Survived
plt.legend(["Not Survived", "Survived"])

# Display the plot
plt.show()
```

5.2.7. Box Plot for Age Distribution

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset across different passenger classes. The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Age by Pclass"**.
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Age'**.

The Titanic dataset contains columns as shown below,

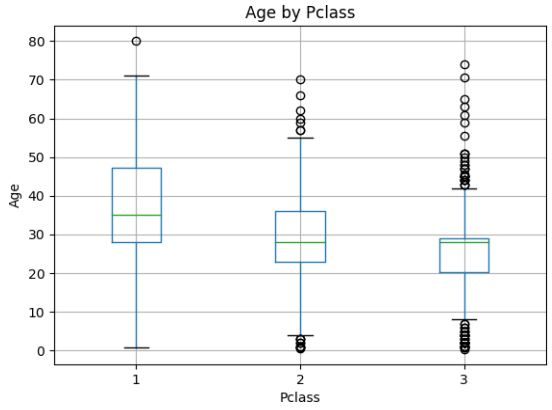
PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Create the boxplot for Age by Pclass
data.boxplot(column='Age', by='Pclass')

# Customize titles and labels
plt.title("Age by Pclass")
```

```
plt.suptitle('') # Removes the default "Boxplot grouped by Pclass"
subtitle
plt.xlabel("Pclass")
plt.ylabel("Age")

# Display the plot
plt.show()
```

5.2.8. Box Plot for Age by Survived

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset based on whether passengers survived or not. The boxplot should display the following specifications:

1. Use the **Survived** column to group the data for the boxplot (0 = Did not survive, 1 = Survived).
2. Set the title of the plot to **"Age by Survival"**.
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as **'Survived'** and the y-axis as **'Age'**.

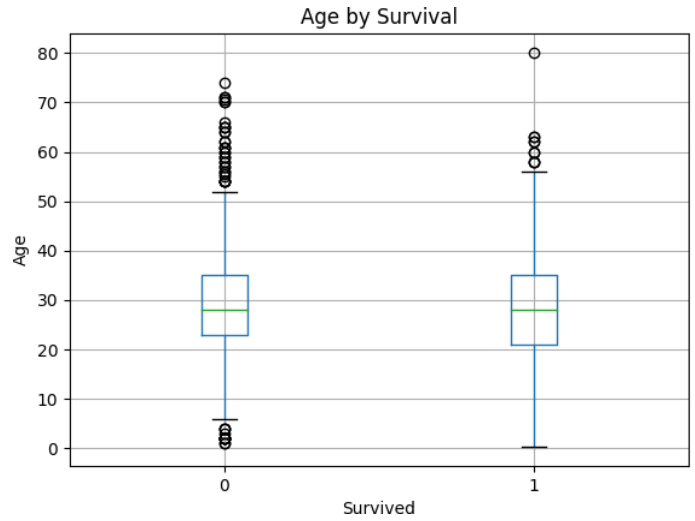
The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Create the boxplot for Age by Survived
data.boxplot(column='Age', by='Survived')

# Customize titles and labels
plt.title("Age by Survival")
plt.suptitle('') # Removes the default "Boxplot grouped by Survived"
subtitle
plt.xlabel("Survived")
plt.ylabel("Age")

# Display the plot
plt.show()
```

5.2.9. Box Plot for Fare by Pclass

Write a Python code to plot a boxplot that shows the distribution of the 'Fare' column from the Titanic dataset based on the passenger class (Pclass). The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Fare by Pclass"**.
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Fare'**.

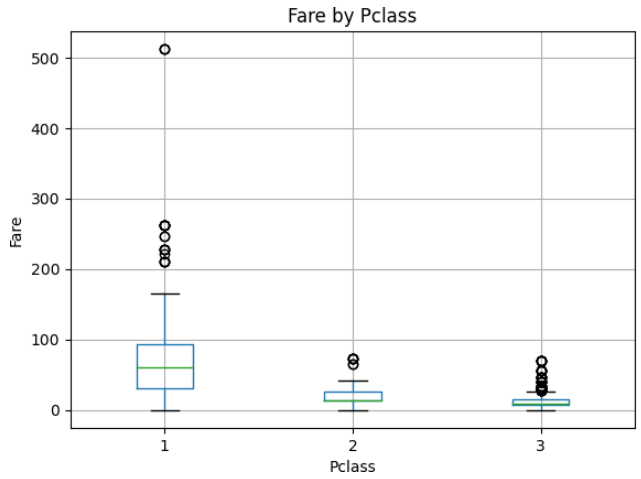
The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Create the boxplot for Fare by Pclass
data.boxplot(column='Fare', by='Pclass')

# Customize titles and labels
plt.title("Fare by Pclass")
plt.suptitle('') # Removes the default "Boxplot grouped by Pclass"
subtitle
plt.xlabel("Pclass")
plt.ylabel("Fare")

# Display the plot
plt.show()
```

5.2.10. Scatter Plot for Age vs Fare

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Set the title of the plot to **"Age vs. Fare"**.
3. Label the x-axis as **'Age'** and the y-axis as **'Fare'**.

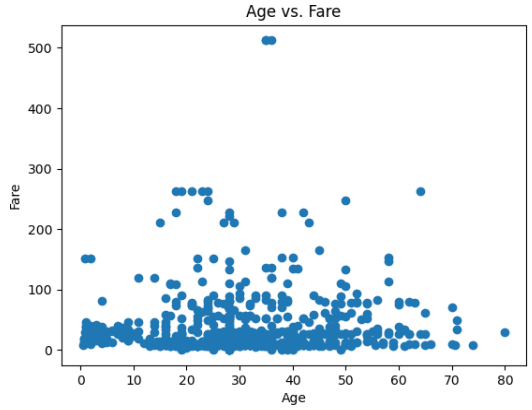
The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
```

```
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Create the scatter plot for Age vs Fare
plt.scatter(data['Age'], data['Fare'])

# Add title and labels
plt.title("Age vs. Fare")
plt.xlabel("Age")
plt.ylabel("Fare")

# Display the plot
plt.show()
```

5.2.11. Scatter Plot for Age vs Fare by Survived

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset, with points color-coded by survival status. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Color the points based on the **Survived** column: **Red** for passengers who did not survive (**Survived = 0**). **Blue** for passengers who survived (**Survived = 1**).
3. Set the title of the plot to **"Age vs. Fare by Survival"**.
4. Label the x-axis as **'Age'** and the y-axis as **'Fare'**.

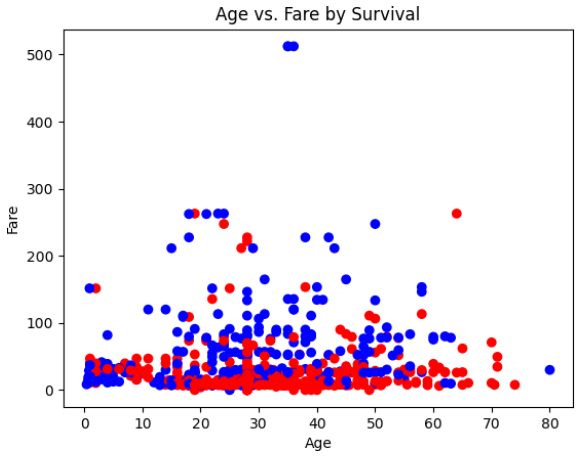
The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

- PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
- 1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
- 2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
- 3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
- 4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
- 5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
- 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
- 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
- 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
- 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
- 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Create a color mapping: 0  > 'red', 1  > 'blue'
```

```
colors = data['Survived'].map({0: 'red', 1: 'blue'})

# Create the scatter plot for Age vs Fare using the c argument for
colors
plt.scatter(data['Age'], data['Fare'], c=colors)

# Add title and labels
plt.title("Age vs. Fare by Survival")
plt.xlabel("Age")
plt.ylabel("Fare")

# Display the plot
plt.show()
```